600086 Final Lab (25-26): High-Performance Shower Simulation

1. Overview

This assessment requires the design, implementation, and critical analysis of a high-performance particle simulation system. You will develop two distinct parallel implementations of a "shower simulation" to explore the architectural paradigms of CPUs and GPUs.

The objective is to simulate shower water flow as individual particles within a 3D environment, managing their kinematic physics, thermal properties, and inter-particle collisions. You must optimize both implementations to handle massive particle counts (100k+) while maintaining system stability and data integrity.

2. Simulation Specification

2.1 The Environment

- Dimensions: A 3D cuboid representing a shower cubicle with dimensions specified below:
  - Height: 2.0 meters
  - Width: 1.0 meter
  - Depth: 1.0 meter
- Emitter (Shower Head): A circular area with a diameter of 0.1m, located at the centre of the cubicle's ceiling.

2.2 Particle Dynamics

- **Spawning:**
  - Particles spawn at random coordinates within the shower head's circular area.
  - Initial Velocity: Particles exit the emitter with a velocity magnitude defined by a global initial speed. The direction is randomized within a cone of -30° to +30° relative to the vertical axis (downward).
- **Motion:** All particles are subject to constant gravitational acceleration.
- **Boundaries:**
  - **Floor:** When a particle hits the floor ($y = 0$), it is removed and immediately respawned at the emitter to maintain a constant active particle count. A global counter must track the total cumulative floor hits.

- **Walls:** If a particle collides with a side wall, you must implement one of the following:
  1. **Reflective:** The particle bounces off the wall (elastic collision).
  2. **Toroidal (Wrapping):** The particle exits one side and re-enters from the opposite side.
  3. **Sticky (Gliding):** The particle "sticks" to the wall (elasticity = 0). It loses its horizontal velocity component but continues to move vertically along the wall surface under the influence of gravity until it reaches the floor.

2.3 Particle Interactions (Collisions)

- **Collision Logic:** Particles are dynamic entities with a baseline mass of 1.0. If two particles occupy the same space, they **merge** into a single, larger particle, with new mass $m_{new} = m_0 + m_1$.
- **Conservation of Momentum: Conservation of Momentum (Standard):** For particles in free-fall, the new velocity vector v_{new} is calculated using the inelastic collision formula:

$$v_{new} = \frac{m_0 v_0 + m_1 v_1}{m_0 + m_1}$$

- **Boundary Constraints (Sticky Mode Only):**
  - If the simulation is running in "Sticky" mode and a collision involves a particle currently gliding on a wall (where horizontal velocity is 0), the resulting merged particle must remain on the wall.
  - The standard momentum formula is applied only to the vertical component (v_y):

$$v_{new,y} = \frac{m_0 v_{0,y} + m_1 v_{1,y}}{m_0 + m_1}$$

  - The horizontal velocity components $v_x, v_z)$ must be forcibly zeroed to prevent the particle from detaching:

$$v_{new,x} = 0, \quad v_{new,z} = 0$$

  - *Note: This effectively simulates the wall absorbing the horizontal momentum of the incoming particle.*

2.4 Thermodynamics (Cooling)

- **Temperature Model:** Particles initialize with a temperature of **1.0 (Hot)** and cool toward 0.0 (Cold).
- **Cooling Formula:**

$$T_{new} = T_{current} - \left(\frac{\Delta t \cdot K_{cool}}{m} \right)$$

- Δt: Time elapsed since the last update.
- K_{cool} : A global cooling factor (tuneable).
- m: Particle mass.
- *Note: Larger particles (higher mass) must cool down more slowly than smaller ones.*

3. Implementation Requirements

You must deliver two separate, fully functional implementations.

3.1 CPU Implementation (Rust)

- **Language: Rust** (Required for its memory safety guarantees and concurrency primitives).
- **Concurrency Model:** The system must utilize at least 6 concurrent threads, strictly partitioned by functionality:
  - Physics/Dynamics: implement on at least 2 dedicated threads to position/velocity updates.
  - Collision Detection and response: implement on at least 2 dedicated threads to collision resolution.
  - Thermodynamics: implement on at least 2 dedicated threads to temperature calculations.
- **Synchronization:** You must demonstrate the use of safe concurrency primitives to manage shared state without race conditions.

3.2 GPU Implementation (CUDA)

- **Language:** C/C++ with CUDA.
- **Architecture:** Leverage massive parallelism utilizing the GPU's many-core architecture.
  - Physics/Dynamics: Implemented as 1 thread per particle.
  - Collision Detection and response**:** Implemented on 1 or more threads.
  - Thermodynamics: Implemented as 1 thread per particle.

4. Visualization & Tuning

4.1 OpenGL Rendering

- **Requirement:** Real-time visualization using OpenGL.
- **Visual Modes:** Toggle between these modes via keyboard input:

1. **Thermal Mode:** Colour gradient from Red (Hot) to Blue (Cold) based on temperature.
2. **Mass Mode:** Distinct colours representing different mass tiers.

- **Performance Scaling:**
  An essential part of this assessment is to analyse the performance characteristics of your simulation. Gradually increase the number of particles in your system until the simulation can no longer maintain smooth particle motion.

  While your simulation should be capable of handling 100,000 or more particles, the graphical rendering component will likely become a bottleneck. To address this limitation, adjust the visualization so that only a subset of particles is displayed. For example, render one particle out of every 100. This approach allows you to observe the simulation without experiencing a significant drop in frame rate caused by graphical constraints.

4.2 Tuning Goals

- **Dynamics:** Tune initial speed such that particles spread significantly, but wall collisions remain infrequent.
- **Thermodynamics:** Tune cool factor K_{cool} so that baseline mass (1.0) particles reach "Cold" (near 0.0) just as they hit the floor. Larger merged particles should remain warmer upon impact.

5. Deliverables & Assessment

The following submissions must be uploaded via Canvas.

5.1 Lab Book Entries

A technical report (approx. 2000 words total) containing:

1. **GPU Design (500 words):** Detail your CUDA kernel design, memory hierarchy usage (shared vs. global), and synchronization strategy.
2. **CPU Design (500 words):** Explain your Rust concurrency model, thread pooling strategy, and how you ensured data safety.
3. **Performance Analysis (500 words):**
   - Compare CPU vs. GPU frame execution times.
   - Identify the maximum stable particle count for each implementation.
   - Analyse the impact of different thread block sizes or partitioning strategies.
4. **Reflection (500 words):** Critical review of challenges faced (e.g., race conditions, debugging overhead) and potential future optimizations.

5.2 Video Demonstration

A narrated video demonstrating:

- Both CPU and GPU implementations running.
- The effects of changing visualization modes (Temperature vs. Mass).
- Evidence of particle merging and cooling.
- Performance mode enabled with high particle counts.

5.3 Software

- **Source Code:** Full Visual Studio solution or project files (Rust & CUDA).
- **Executables:** Working binaries or reliable build scripts.